

SAVITRIBAI PHULE PUNE UNIVERSITY

Semantic Search Engine Architecture

Dense Sentence Embeddings, FAISS Nearest-Neighbor Indexing, and Information Retrieval Benchmarking vs TF-IDF

Honours Course in Agentic AI | Course: HAAI301COM & HAAI302COM (Unit-III: Vector Databases & Embedding)

The Evolution of Information Retrieval Paradigms

Era 1: Exact Boolean Matching

- Exact string and keyword matching (AND, OR, NOT).
- Binary membership: A document either contains the token or does not.
- Zero ranking: All matching documents treated with equal weight.
- High vulnerability to typographical errors and morphological changes.

Era 2: Statistical Lexical (TF-IDF)

- Frequency-based scoring: Term Frequency and Inverse Document Frequency.
- Vector Space Model: Documents represented as sparse bag-of-words vectors.
- Cosine similarity ranking based on term overlap weights.
- Limitation: Incapable of understanding synonyms, context, or semantic intent.

Era 3: Dense Semantic Vector Search

- Continuous vector representations learned by deep Transformer bi-encoders.
- Encodes semantic intent, conceptual relationships, and domain analogies.
- Sub-millisecond approximate nearest neighbor (ANN) retrieval via FAISS.
- Foundational substrate for modern RAG and Agentic AI memory.

Classical Keyword Search: TF-IDF & Inverted Index Mechanics

Mathematical Formulation

- Term Frequency (TF): Measures localized frequency of term t inside document d : $TF(t, d) = f(t, d) / \sum(f(t', d))$.
- Inverse Document Frequency (IDF): Dampens common corpus-wide words: $IDF(t, D) = \ln((1 + |D|) / (1 + |d \in D : t \in d|)) + 1$.
- TF-IDF Weight: $w(t, d) = TF(t, d) * IDF(t, D)$.
- Cosine Similarity: $\text{dot_product}(v_q, v_d) / (||v_q|| * ||v_d||)$.

Inverted Index Architecture

- Posting Lists: Map unique vocabulary terms to the list of document IDs containing that term.
- Extreme Sparsity: Vocabulary size $|V|$ often exceeds 100,000 terms; individual document vectors are 99.9% zeros.
- Computational Efficiency: Fast integer posting list intersection enables sub-millisecond keyword lookup.
- Memory Footprint: Scalable inverted indexes run efficiently on commodity CPUs.

The Lexical Mismatch Problem: Why Keyword Search Fails

Synonymy & Vocabulary Mismatch

- Vocabulary Mismatch: The searcher and the document author use different surface words for identical concepts.
- Synonymy Failure: Searching 'myocardial infarction' fails to match clinical records describing 'heart attack'.
- Zero Overlap: If no verbatim word overlaps exist, TF-IDF produces an exact cosine score of 0.0000.
- Paraphrase Vulnerability: Descriptive queries ('methods to stop models from making up facts') fail against technical texts ('eliminate hallucinations').

Polysemy & Context Insensitivity

- Polysemy: A single surface word possesses multiple meanings depending on context (e.g. 'Apple' company vs fruit; 'bank' financial vs river).
- Bag-of-Words Limitation: Word ordering and syntax are discarded: 'dog bites man' produces the identical vector as 'man bites dog'.
- Negation Blindness: 'not bad' is treated identically to 'bad' in standard unigram bag-of-words models.
- Result: High false-negative rates on natural, conceptual search queries.

Dense Semantic Search: Bi-Encoder Sentence Transformers

Bi-Encoder Architecture (Reimers & Gurevych, 2019)

- Twin Encoders: Queries and documents are mapped independently into a continuous vector space $\mathbb{R}^d$.
- SentenceTransformer Model: 'all-MiniLM-L6-v2' maps sentences to fixed 384-dimensional dense vectors.
- Mean Pooling: Applies contextual multi-head self-attention across tokens, then pools hidden states into a single vector.
- Asymmetric Search: Pre-indexes millions of documents offline; encodes the query online in single-digit milliseconds.

Semantic Hypersphere Geometry

- Geometric Semantics: Synonyms and semantically aligned sentences map to neighboring coordinates.
- Cross-Lingual Capabilities: Understands multilingual concepts even when languages differ.
- Dense Information Density: Every dimension in the 384-dim vector carries continuous semantic signal (no zero sparsity).
- Preserves Context: Captures nuance, prepositional relationships, and conceptual intent.

Vector Similarity Metrics & Geometric Equivalence

Distance & Similarity Metrics

- Cosine Similarity: $\cos(u, v) = (u \cdot v) / (||u|| * ||v||)$. Measures angular orientation regardless of magnitude.
- Inner Product (Dot Product): $IP(u, v) = \sum(u_i * v_i)$. Sensitive to both angle and vector magnitude.
- Euclidean Distance (L2): $d(u, v) = \sqrt{\sum((u_i - v_i)^2)}$. Measures straight-line geometric distance.
- Triangle Inequality: L2 distance obeys triangle inequality, enabling space partitioning trees.

The Mathematical Proof of L2 Normalization

- Pre-Normalization: When vectors are normalized such that $||u|| = 1$ and $||v|| = 1$:
- $\cos(u, v) = (u \cdot v) / (1 * 1) = u \cdot v$ (Inner Product).
- Euclidean Link: $||u - v||^2 = ||u||^2 + ||v||^2 - 2(u \cdot v) = 2 - 2 * \cos(u, v)$.
- Engineering Impact: Searching via FAISS IndexFlatIP computes exact cosine similarity via fast dot products without computing square roots at query time.

Facebook AI Similarity Search (FAISS) System Architecture

Why FAISS is the Industry Standard

- High-Throughput C++ Core: Developed by Meta AI Research for billion-scale similarity search.
- SIMD Hardware Acceleration: Utilizes AVX-512 and SSE vector registers for parallel dot-product execution.
- GPU & CPU Support: Native support for multi-GPU streaming and memory-mapped disk storage.
- Hardware Efficiency: Up to 8.5x faster than standard Python / NumPy matrix multiplication routines.

Memory Layout & Cache Optimization

- Contiguous Row-Major Storage: Vectors are stored contiguously in memory (C-order) to maximize CPU L1/L2 cache hits.
- Batch Query Processing: Queries processed in parallel matrix-matrix multiplication (GEMM) blocks.
- Zero Copy Python Bindings: NumPy arrays pass directly into C++ pointer memory without serialization overhead.
- Extensible: Serves as the indexing engine behind vector databases like Milvus, Chroma, and LangChain.

FAISS Index Taxonomy: Exact vs Approximate (ANN) Structures

IndexFlatIP / IndexFlatL2

- Exhaustive Brute-Force Search: Computes distances to all N vectors.
- Recall: Exact 100% ground-truth accuracy.
- Complexity: $O(N * d)$.
- Best For: Small to medium corpora (< 100,000 vectors) where absolute recall is mandatory.

IndexIVFFlat (Inverted File)

- Clustering Space: Uses k-means to divide space into C Voronoi cells.
- Approximate Querying: Compares query only against closest nprobe centroids.
- Speedup: Searches only a fraction of vectors ($N / C * nprobe$).
- Best For: Scalable search up to millions of vectors with 95%+ recall.

IndexHNSW (Navigable Graphs)

- Hierarchical Small World Graphs: Multi-layer graph routing.
- Logarithmic Complexity: $O(\log N)$ search latency.
- State of the art ANN: Unbeatable speed-accuracy trade-off.
- Trade-off: Higher RAM consumption to store graph edge lists.

Information Retrieval (IR) Quality Metrics Formulation

Precision@k

- Formula: $\text{Precision@k} = |\text{Relevant in Top } k| / k$.
- Measures retrieval purity: Fraction of returned results that are genuinely relevant.
- Example: If 2 of top-3 results are relevant, $\text{Precision@3} = 2/3 = 0.67$.
- Focus: Minimizing noise in the retrieved set.

Recall@k

- Formula: $\text{Recall@k} = |\text{Relevant in Top } k| / |\text{Total Ground Truth}|$.
- Measures retrieval completeness: Fraction of all relevant documents found.
- Example: If corpus has 1 relevant document and it is in top-3, $\text{Recall@3} = 1.00$.
- Focus: Preventing missing critical facts.

Mean Reciprocal Rank (MRR)

- Formula: $\text{MRR} = (1/|Q|) * \sum (1 / \text{rank}_i)$.
- Evaluates ranking position of the first relevant document.
- Rank 1 -> 1.00 | Rank 2 -> 0.50 | Rank 3 -> 0.33 | Miss -> 0.00.
- Critical for RAG: Top-ranked chunks receive highest attention in LLM prompts.

Quantitative Benchmark: Traditional TF-IDF vs Dense FAISS

Experimental Setup & Results

- Corpus: 12 technical domain documents across AI, systems, and security.
- Query Suite: Mix of exact technical phrases and non-overlapping conceptual paraphrases.
- Lexical Match Query: Both TF-IDF and FAISS succeed with Precision@3 = 0.33 and Recall@3 = 1.00.
- Synonym/Paraphrased Query: TF-IDF scores 0.0000 and misses target completely (Recall = 0.00).
- FAISS Dense Retrieval: Consistently maintains high cosine similarity (>0.60) and ranks target at Rank 1.

Latency & Resource Comparison

- TF-IDF Search Latency: ~1.5 ms (Fast sparse matrix dot products on CPU).
- FAISS Dense Search Latency: ~0.2 ms for index lookup + ~8.0 ms for query embedding encoding.
- Memory Footprint: TF-IDF stores sparse CSR matrix; FAISS stores continuous float32 array (384 * 4 bytes per doc).
- Conclusion: The slight neural encoding latency overhead is vastly outweighed by the elimination of vocabulary mismatch.

Production Architecture: Hybrid Search & Reciprocal Rank Fusion

Why Neither Model Alone is Sufficient

- Dense Search Failure: Struggles with exact alphanumeric serial numbers, product SKUs, and rare acronyms.
- Sparse Search Failure: Fails on natural language questions, synonyms, and multilingual queries.
- Solution: Hybrid Search combining BM25 sparse keyword retrieval with Dense FAISS vector retrieval.
- Dual Pipeline: Ingests documents into both an inverted index and a vector database simultaneously.

Reciprocal Rank Fusion (RRF)

- Non-parametric ranking combination without calibrating raw score scales:
- $RRF\ Score(d) = \sum (1 / (k_rrf + rank_sparse(d)) + 1 / (k_rrf + rank_dense(d)))$.
- Standard constant: $k_rrf = 60$.
- Documents favored by both keyword exactness and semantic relevance rise to the top ranks.
- Industry Standard: Deployed in Elasticsearch, Pinecone, Qdrant, and Azure AI Search.

Key Takeaways for AI Engineers & Viva Voce Reference

Core Principles to Master

- Vocabulary Mismatch is real: Never rely solely on keyword matching for user-facing semantic queries.
- L2 Normalization is mandatory: Always unit-normalize embeddings before IndexFlatIP to obtain true cosine similarity.
- Indexing Trade-offs: Use Flat indexes for exactness under 100k vectors; use IVF or HNSW for million-scale workloads.
- Evaluate with Multiple Metrics: Track Precision@k for purity, Recall@k for coverage, and MRR for ranking quality.

Connection to Agentic AI Curriculum

- RAG Pipelines (Exp 4): FAISS and Chroma provide the semantic retrieval layer feeding context into LLMs.
- Long-Term Memory (Exp 6 & 7): Autonomous agents use vector indexing for episodic and semantic memory recall.
- Tool Discovery (Exp 4 & 5): Agents search across hundreds of API tool definitions using semantic embeddings.
- Lab Notebook: Refer to `Experiment_05_Semantic_Search_FAISS_TFIDF.ipynb` for complete hands-on execution.